CS315 · ALGORITHMS ANALYSIS & DESIGN

# Midterm 462 الحل

25 درجة

**محارب**

## 01 Q1: Linear probing حل

| Key | key mod 7 | Probes | Cell |
|---|---|---|---|
| **76** | 6 | 6 | **6** |
| **93** | 2 | 2 | **2** |
| **40** | 5 | 5 | **5** |
| **47** | 5 | 5 → 6 → 0 | **0** |
| **10** | 3 | 3 | **3** |
| **55** | 6 | 6 → 0 → 1 | **1** |

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 47 | 55 | 93 | 10 | | 40 | 76 |

```
47:  h = 47 mod 7 = 5 taken   →   6 taken   →   (6 + 1) mod 7 = 0
```

```
55:  h = 55 mod 7 = 6 taken   →   0 taken   →   1
```

لاحظ اللفّة من آخر الجدول (6) إلى أوله (0).

## 02 حل Q2: True / False + MCQ

| # | الجواب | السبب |
|---|---|---|
| 1 | **a. True** | الحرف اللي يتكرر أكثر ياخذ code أقصر. |
| 2 | **b. False** | الـ greedy ما يعطي الحل الأمثل في كل مسائل الـ DP. |
| 3 | **c** | Dynamic programming يخزن نتائج الـ sub-problems ويستخدمها. |
| 4 | **b. False** | المعادلة الصحيحة آخرها `T(MERGE)` مو $T(\frac{MERGE}{2})$. |
| 5 | **c. Other value** | الوقت `O(n)` لما الأنشطة مرتبة حسب وقت الانتهاء. |
| 6 | **a. True** | الـ DP يحل كل sub-problem مرة وحدة ويخزن نتيجتها، فيوفر وقت كثير. |
| 7 | **d** | little-omega = Strictly greater. الـ Greater than or equal هي Big-Omega. |
| 8 | **C. O(n²)** | لوبين متداخلين، كل واحد يلف `n` مرة. |
| 9 | **A. O(1)** | أفضل حالة: العنصر في أول خانة. |
| 10 | **b** | Binary search يقسم المدى على 2 كل مرة. |

## 03 حل Q3: Greedy Knapsack

الـ greedy يرتب الأصناف حسب القيمة لكل كيلو (value ÷ weight) من الأكبر للأصغر، وياخذ بالترتيب لين

يمتلي الكيس.

| Item | Weight (kg) | Value (SAR) | Value per kg | Taken |
|---|---|---|---|---|
| **Saffron** | 2 | 4000 | 2000 | 2 kg → 4000 |
| **Cardamom** | 2 | 1000 | 500 | 2 kg → 1000 |
| **Cinnamon** | 5 | 500 | 100 | 1 kg → 100 |
| **Pepper** | 10 | 500 | 50 | 0 |

```
Weight used = 2 + 2 + 1 = 5 kg
```

```
Total value = 4000 + 1000 + 100 = 5100 SAR
```

وين المشكلة؟ لو لازم ناخذ الصنف كامل (knapsack 1-0)، الـ greedy ياخذ Saffron و Cardamom ويوقف عند 4 كيلو و 5000 ريال، ويبقى كيلو فاضي ما يقدر يعبيه. الحل: نسمح بأخذ جزء من الصنف (**Fractional knapsack**)، فناخذ كيلو واحد من Cinnamon ويصير المجموع 5100.

## 04 حل Q4: LCS

```
X[i] = Y[j]:   c[i, j] = c[i-1, j-1] + 1        diagonal + 1
```

```
X[i] ≠ Y[j]:   c[i, j] = max(top, left)
```

| | ⌀ | A | B | C | A | D | F |
|---|---|---|---|---|---|---|---|
| ⌀ | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| C | 0 | 1 | 1 | 2 | 2 | 2 | 2 |
| B | 0 | 1 | 2 | 2 | 2 | 2 | 2 |
| A | 0 | 1 | 2 | 2 | 3 | 3 | 3 |
| E | 0 | 1 | 2 | 2 | 3 | 3 | 3 |
| D | 0 | 1 | 2 | 2 | 3 | 4 | 4 |

```
Length = 4
```

```
LCS = ACAD        (ABAD is also a correct answer)
```

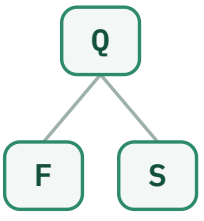
الخانات الملونة هي مسار الرجوع من آخر خانة: عند كل تطابق ناخذ الحرف ونطلع قطري.

## 05 حل Q5: B-tree

B-tree من degree 3: كل node فيها 3 أبناء على الأكثر، يعني مفتاحين على الأكثر. لما node توصل 3 مفاتيح تنقسم: المفتاح الأوسط يطلع للأب، والأول والثالث يصيرون nodes منفصلة.

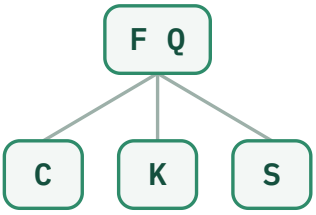
### 1 F, S, Q

`F S` ثم Q يخليها `F Q S` : ثلاث مفاتيح، فتنقسم و Q يطلع.



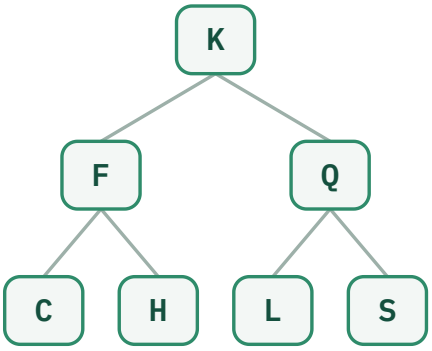
**2** **K, C**

`F K` ثم C يخليها `C F K` ، فتنقسم و F يطلع للـ root.

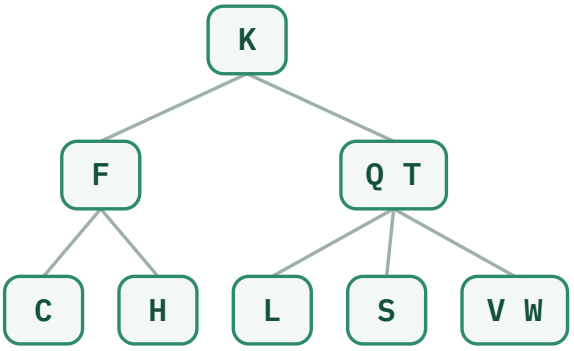



**3** **L, H**

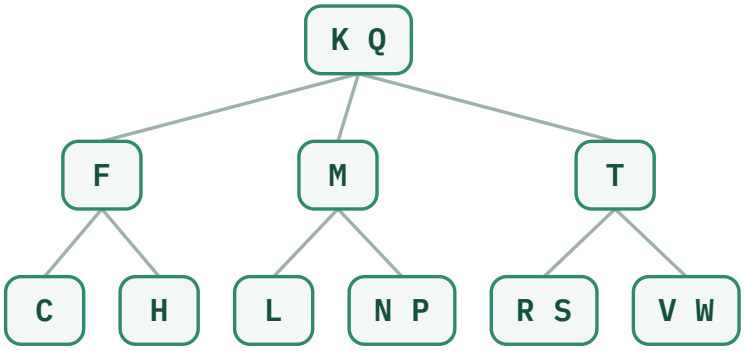
`K L` ثم H يخليها `H K L` : K يطلع، فالـ root يصير `F K Q` وينقسم هو بعد، و K يصير root جديد.



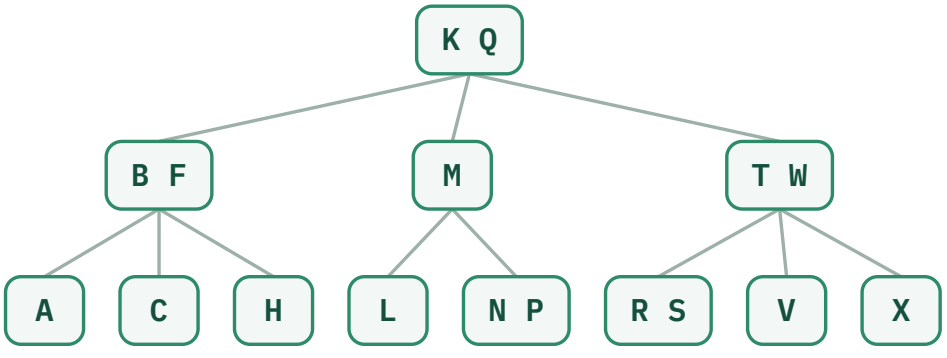
**T, V, W** 4



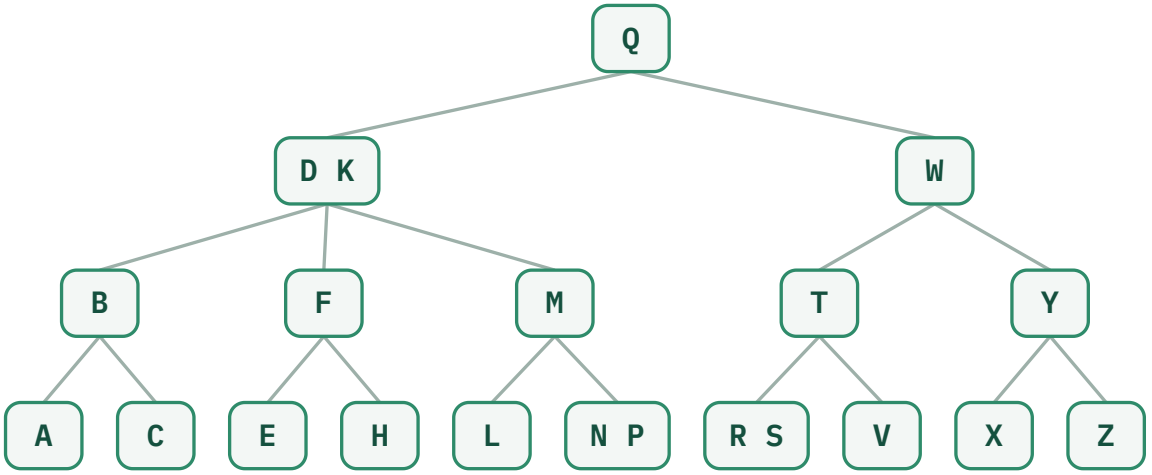

**M, R, N, P** 5




**A, B, X** 6



**7** الشكل النهائي · Y, D, Z, E




الشكل النهائي: root واحد، تحته nodeين، تحتهم خمس، وعشر أوراق.